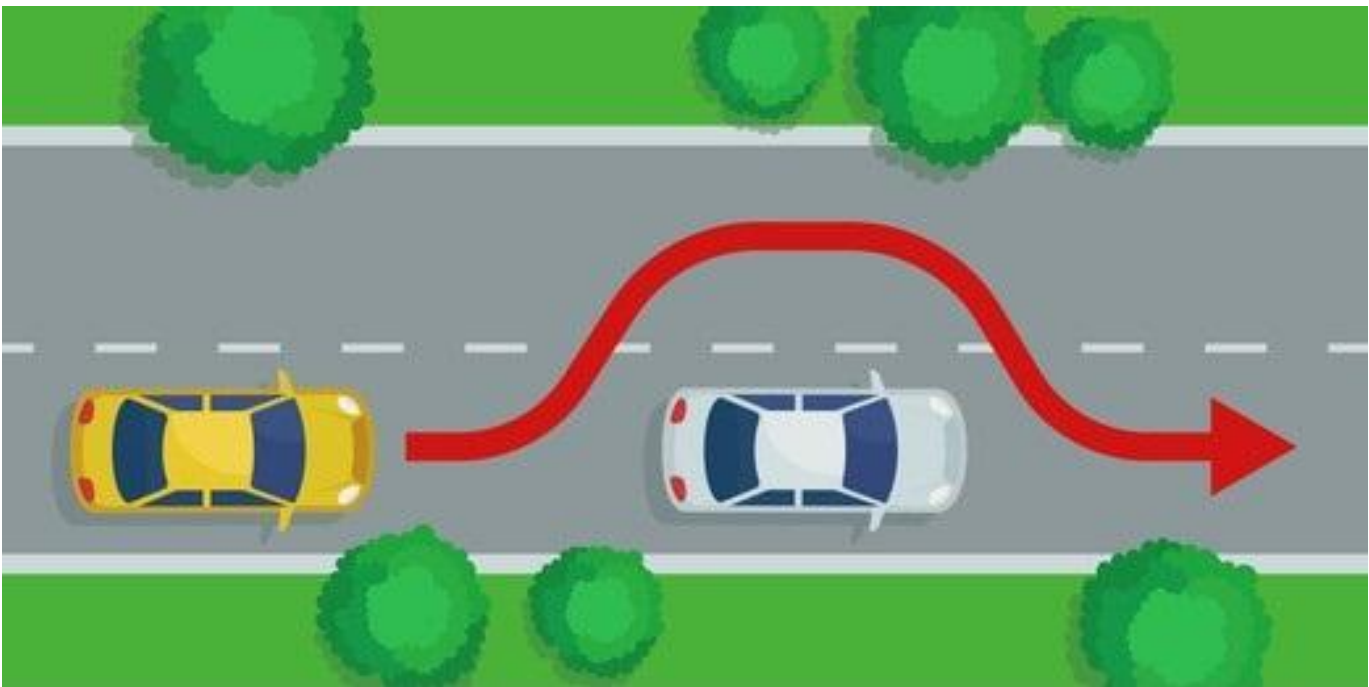


Chapter 4A: Adversarial Search and Games (Part 1)

AMCS2104 Fundamentals of Artificial Intelligence

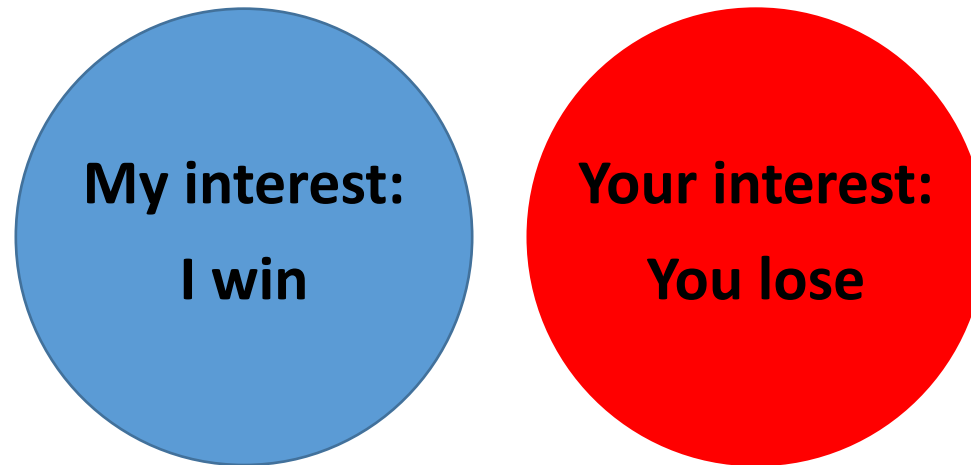
Content

- Adversarial Search Problems
- Games
- Formulating Games
- Game Tree
- Optimal Strategy
- The Minimax Algorithm
- Multiplayer Games
- Alpha-Beta Pruning



Adversarial Search Problems

- In a competitive environment, agents' **goals are in conflict**.



- When they compete with each other, it gives rise to **adversarial search problems**, which is often known as **games**.

Games

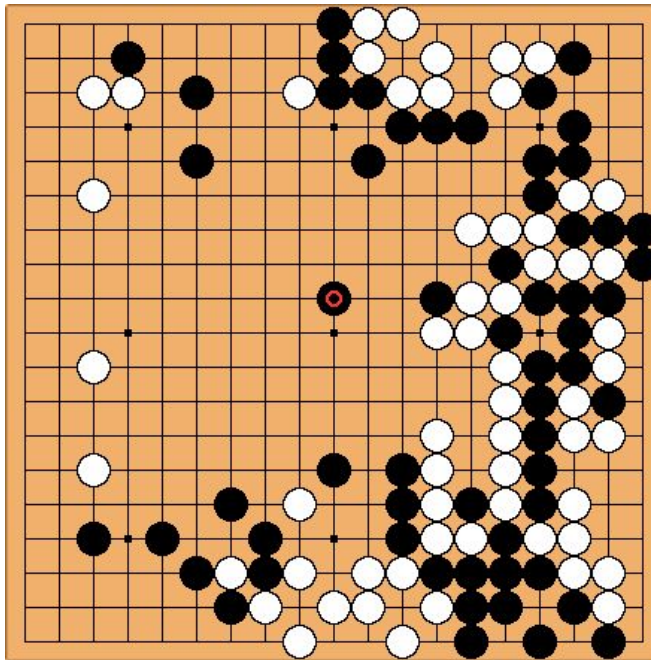
- For games in A.I.:
 - A **move** refers to an **action**.
 - A **position** refers to a **state**.
- The most commonly studied games in A.I. are:
 - Turn-taking
 - Two-player
 - Perfect information (i.e., a synonym for fully observable.)
 - Deterministic
 - Zero-sum
 - What is good for one player is just as bad for the other.

Two-Player Zero-Sum Games

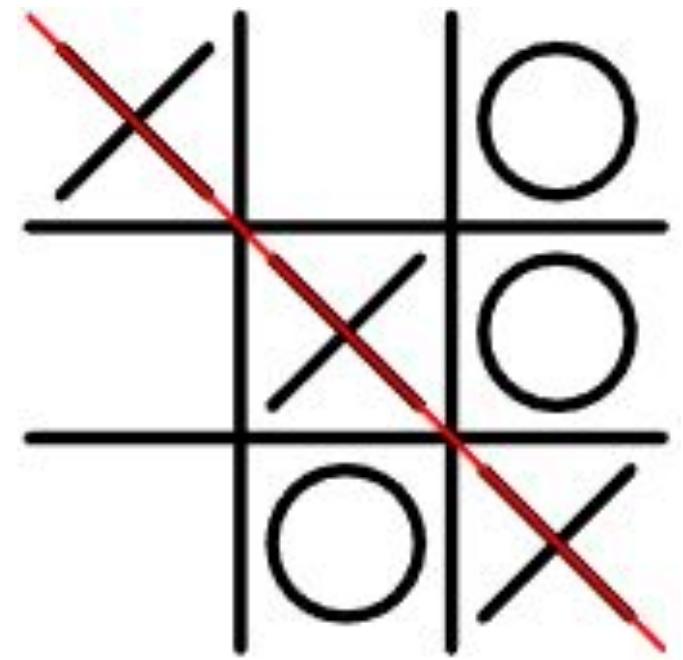
□ The following games will be used to explain how adversarial search works.



Chess



Go



Tic-Tac-Toe

Problem Formulation

- Two-players
 - Players are referred to as MAX and MIN.
 - MAX moves first, followed by MIN, and then it repeats until the game is over.
- At the end of the game
 - Points are awarded to the winner.
 - Points are deducted from the loser.

Problem Formulation

- A game can be formally defined with the following elements:
 - s_0
 - The **initial state**, which specifies how the game is set up at the start.
 - **ACTIONS (s)**
 - The set of legal moves in state s .
 - **RESULT (s, a)**
 - The **transition model** which defines the state resulting from taking action a in state s .
 - **TO-MOVE (s)**
 - The player whose turn it is to move in state s .

Problem Formulation

□ IS-TERMINAL (s)

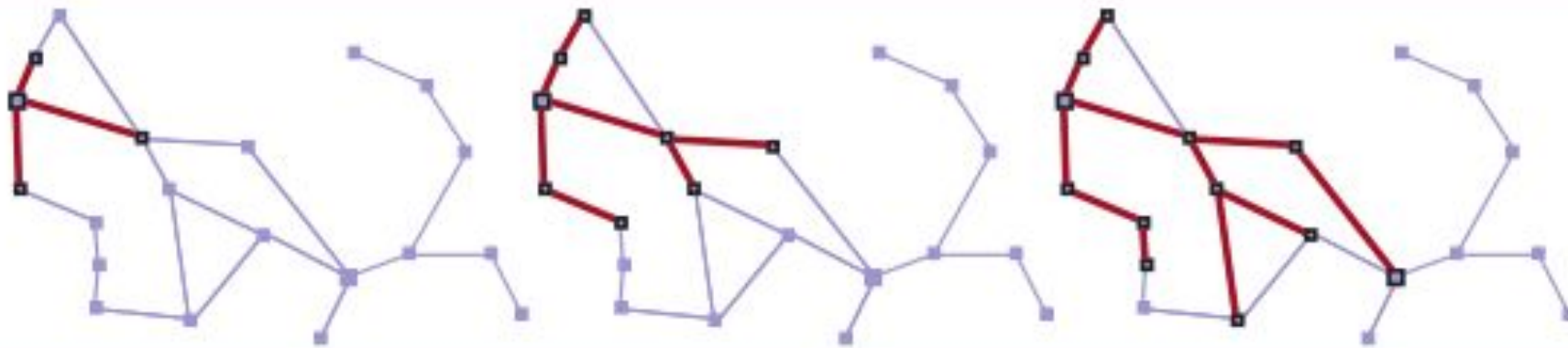
- A **terminal test** which returns **true** when the game is over and **false** otherwise.
- States where the game has ended are called **terminal states**.

□ UTILITY (s, p)

- A utility function (also known as **objective function** or **payoff function**) defines the final numeric value to player p when the game ends in terminal state s .
- Example
 - In chess, the outcome is a win, loss, or draw, with values 1, 0, or 1/2.

Game Tree

- As in Chapter 3...
 - The **state space** of a game is defined by the **initial state**, **ACTIONS** function, and **RESULT** function.
 - We can superimpose a **game tree** over part of the state space graph to determine what move to make.



Game Tree

- A **game tree** is a type of search tree that represents all possible game states as a result of each move made by the players alternatively within a game.
- A **complete game tree** is a game tree that starts at the initial position and follows every sequence of moves all the way to **terminal states**.
- Nodes corresponding to terminal states are called **leaf nodes** or **terminal nodes**.
- A game tree can be thought of as **a technique for analyzing adversarial games**, which determine the actions that player takes to win the game.

Example: Tic-Tac-Toe

□ s_0

□ A setup where all the 9 boxes are empty.

□ **ACTIONS (s)**

□ Place a symbol "X" at any empty box (only by MAX).

□ Place a symbol "O" at any empty box (only by MIN).

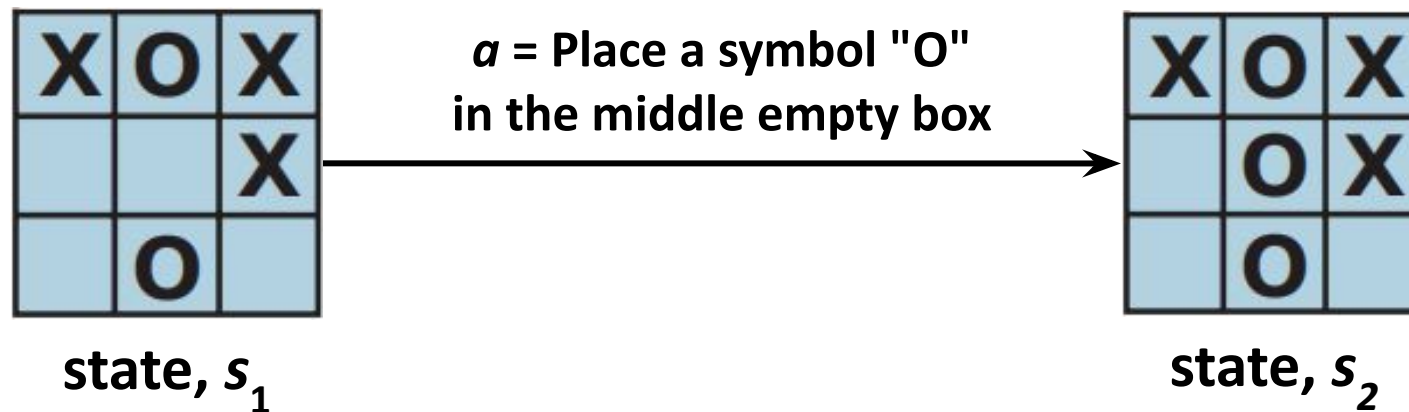
□ **TO-MOVE (s)**

□ MAX and MIN takes turn to make a move, beginning from MAX.

Example: Tic-Tac-Toe

□ **RESULT (s, a)**

- Place a symbol ("X" or "O") into an empty box results in the box occupied.
- E.g., **RESULT (s_1, a) = s_2**



Example: Tic-Tac-Toe

□ IS-TERMINAL (s)

- A state will become terminal when a player has three in a row or all the squares are filled.

□ UTILITY (s, p)

- The utility score is either
 - 1: MAX is the winner
 - 0: It is a draw game
 - -1: MAX is the loser

X	O	X
	O	X
	O	

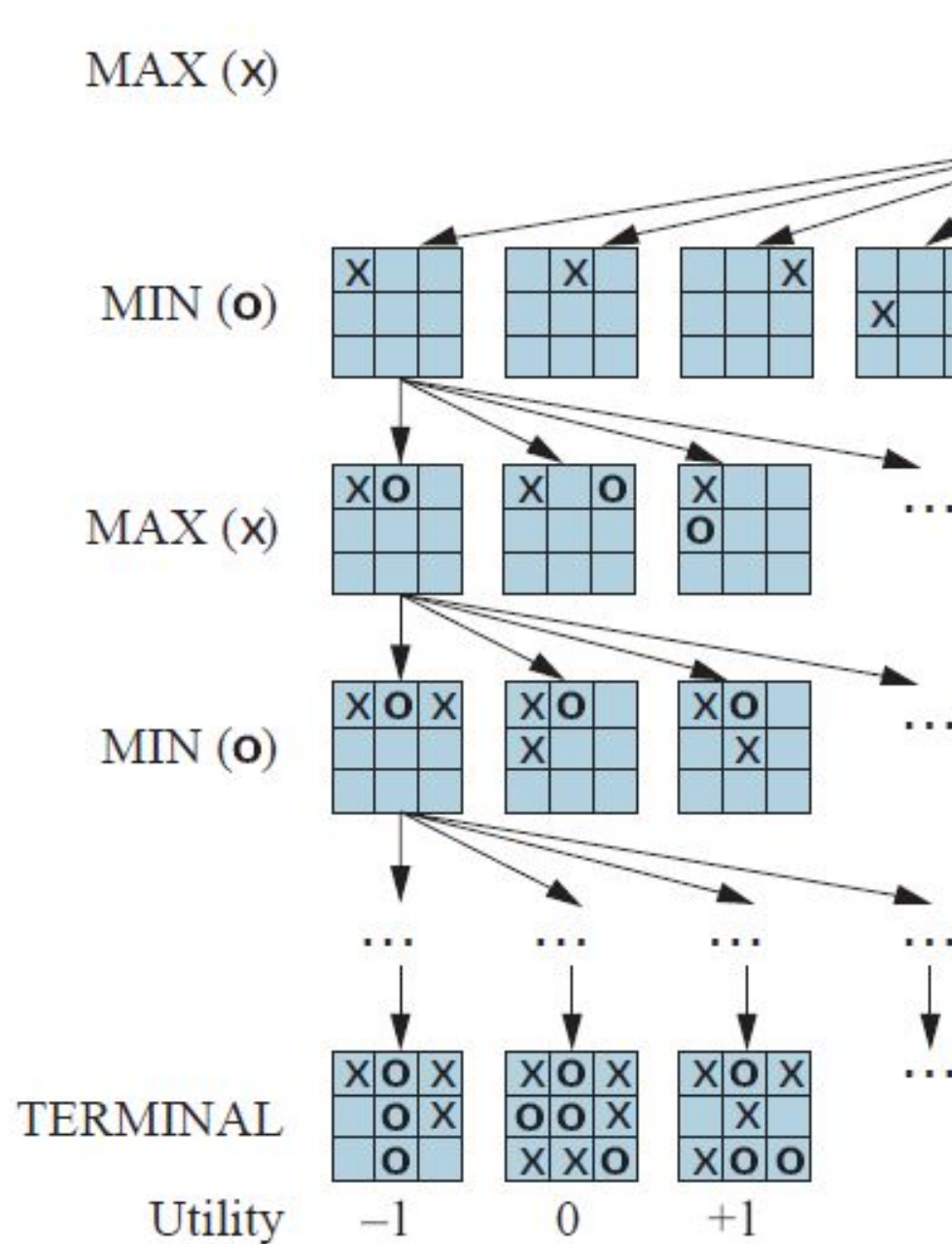
-1

X	O	X
O	O	X
X	X	O

0

X	O	X
	X	
X	O	O

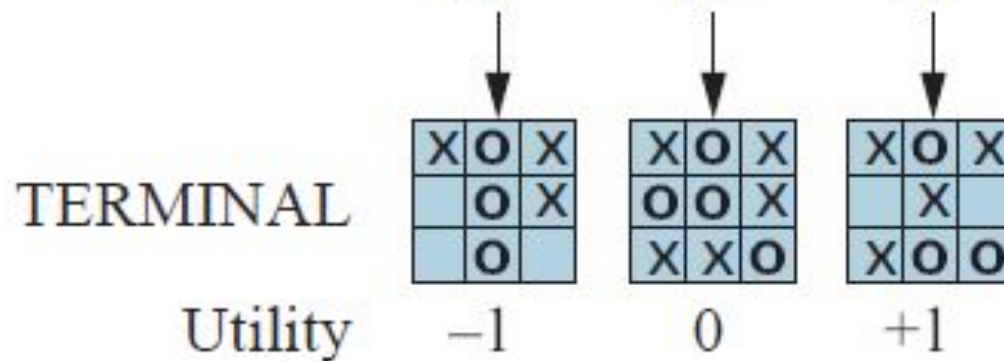
+1



- This is a partial game tree for the game of tic-tac-toe.
- The complete game tree has fewer than $9! = 362,880$ terminal nodes.

MAX vs. MIN

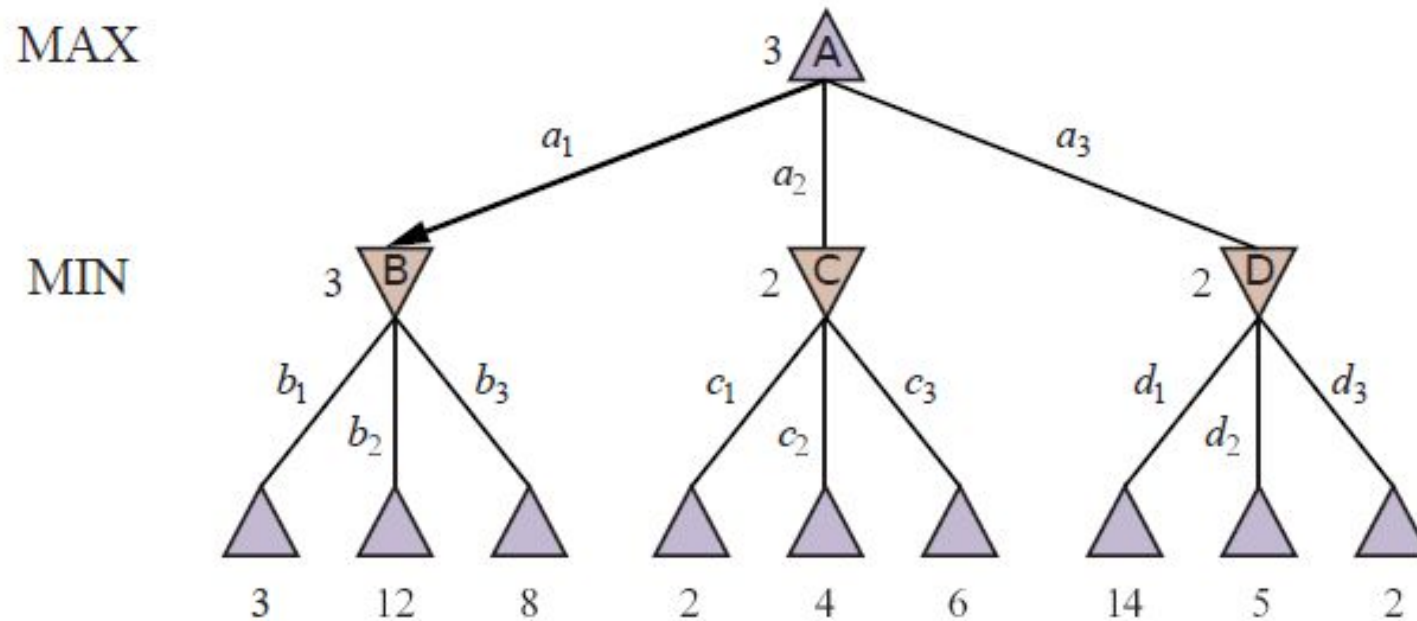
- ❑ **Question:** Why players MAX and MIN are named so?
- ❑ The number on each leaf node indicates the utility value of the terminal state from the point of view of MAX:
 - ❑ High values are good for MAX and bad for MIN.
 - ❑ Low values are bad for MAX and good for MIN.



Conditional Plan for Games

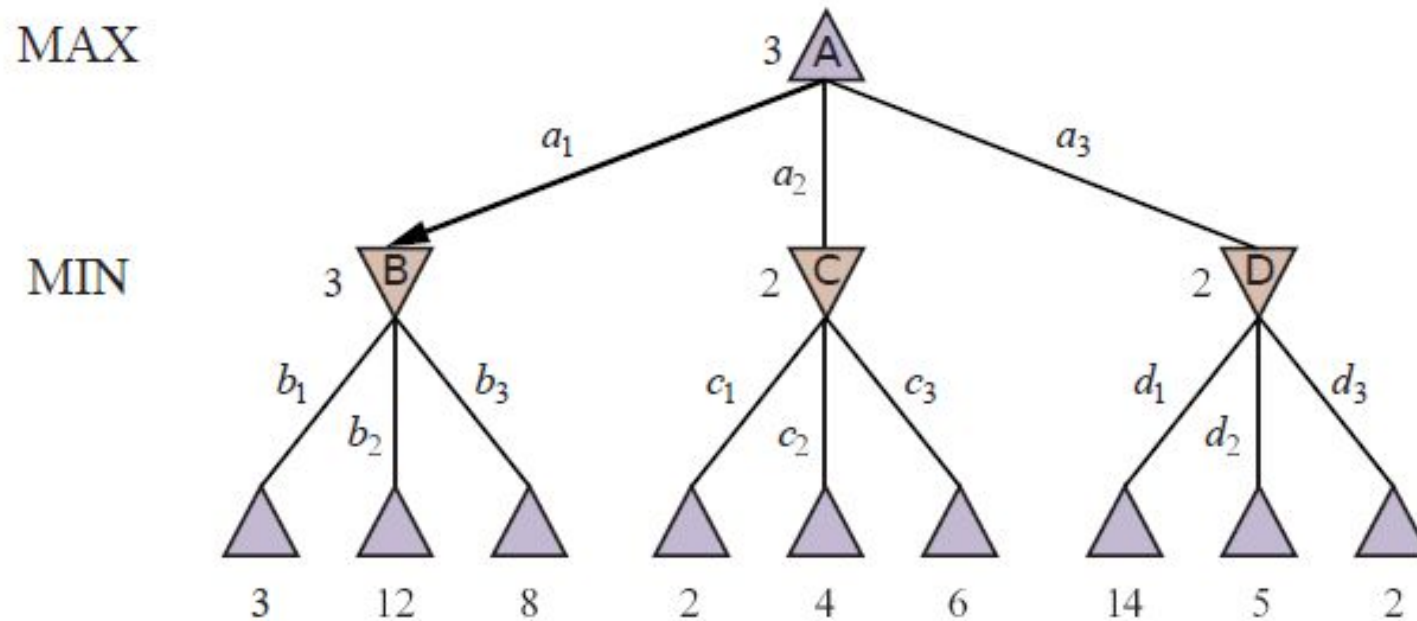
- In a two player game, MAX wants to find a sequence of actions leading to a win, whereas MIN tries to defeat MAX.
- MAX's strategy must be a **conditional plan**, a **contingent strategy** specifying a response to each of MIN's possible moves.
- For games with a binary outcome (win or lose), a **conditional plan** can be generated using **AND-OR search**.
- For games with multiple outcome scores, the **conditional plan** is generated using the **minimax search** algorithm.
- The definition of a winning strategy for the game is identical to the definition of a solution for a nondeterministic planning problem.

A Two-Ply Game Tree



- In some games, the word "**move**" means that **both players have taken an action**.
- Therefore, one **ply** is used to refer to **one move by one player**.
- This particular two-ply game ends after one move each by MAX and MIN.

A Two-Ply Game Tree



- The terminal nodes show the utility values (i.e., a range from 2 to 14) for MAX.
- The other nodes are labeled with their **minimax values**.
- **Question:** What is MAX's best move?

The minimax Value

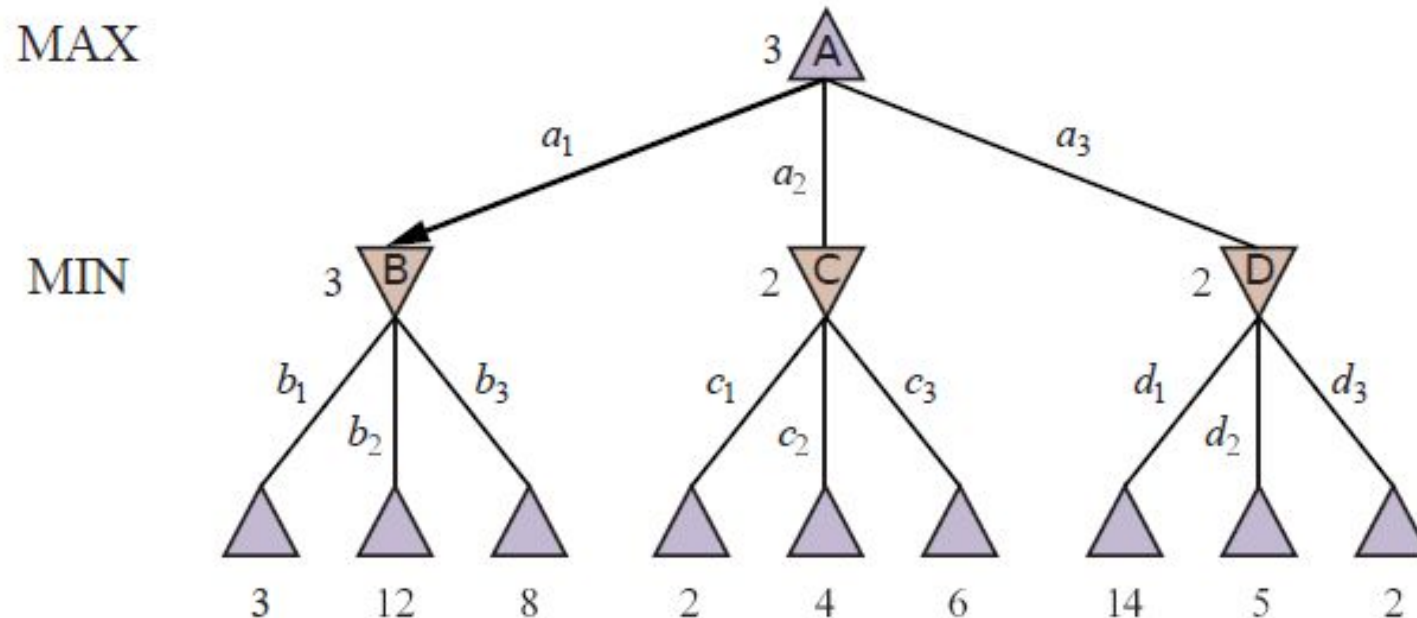
- The optimal strategy can be determined by working out the **minimax value** of each state in the tree, represented by **MINIMAX(s)**:

$$\text{MINIMAX}(s) = \begin{cases} \text{UTILITY}(s, \text{MAX}) & \text{if IS-TERMINAL}(s) \\ \max_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{if TO-MOVE}(s) = \text{MAX} \\ \min_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{if TO-MOVE}(s) = \text{MIN} \end{cases}$$

- The **minimax value** is the utility (for MAX) of being in that state, **assuming that both players play optimally from there to the end of the game.**
- For a terminal state, its minimax value is its utility.

The minimax Value

- **minimax decision:** MAX's best move is a_1 as it leads to the state with the highest minimax value.
- **Question:** What will happen if MIN does not play optimally?



The minimax Algorithm

- The **minimax algorithm** computes the optimal move using minimax based on the assumption that the opponent plays to minimize utility.
- It uses a **simple recursive computation** that proceeds all the way down to the leaves of the tree, and then the minimax values are **backed up** through the tree as **the recursion unwinds**.

function MINIMAX-SEARCH(*game, state*) **returns** *an action*

$\text{player} \leftarrow \text{game.TO-MOVE}(\text{state})$

$\text{value, move} \leftarrow \text{MAX-VALUE}(\text{game, state})$

return *move*

function MAX-VALUE(*game, state*) **returns** *a (utility, move) pair*

if *game.IS-TERMINAL(state)* **then return** *game.UTILITY(state, player), null*

$v, \text{move} \leftarrow -\infty$

for each *a* **in** *game.ACTIONS(state)* **do**

$v2, a2 \leftarrow \text{MIN-VALUE}(\text{game, game.RESULT}(\text{state}, a))$

if $v2 > v$ **then**

$v, \text{move} \leftarrow v2, a$

return v, move

function MIN-VALUE(*game, state*) **returns** *a (utility, move) pair*

if *game.IS-TERMINAL(state)* **then return** *game.UTILITY(state, player), null*

$v, \text{move} \leftarrow +\infty$

for each *a* **in** *game.ACTIONS(state)* **do**

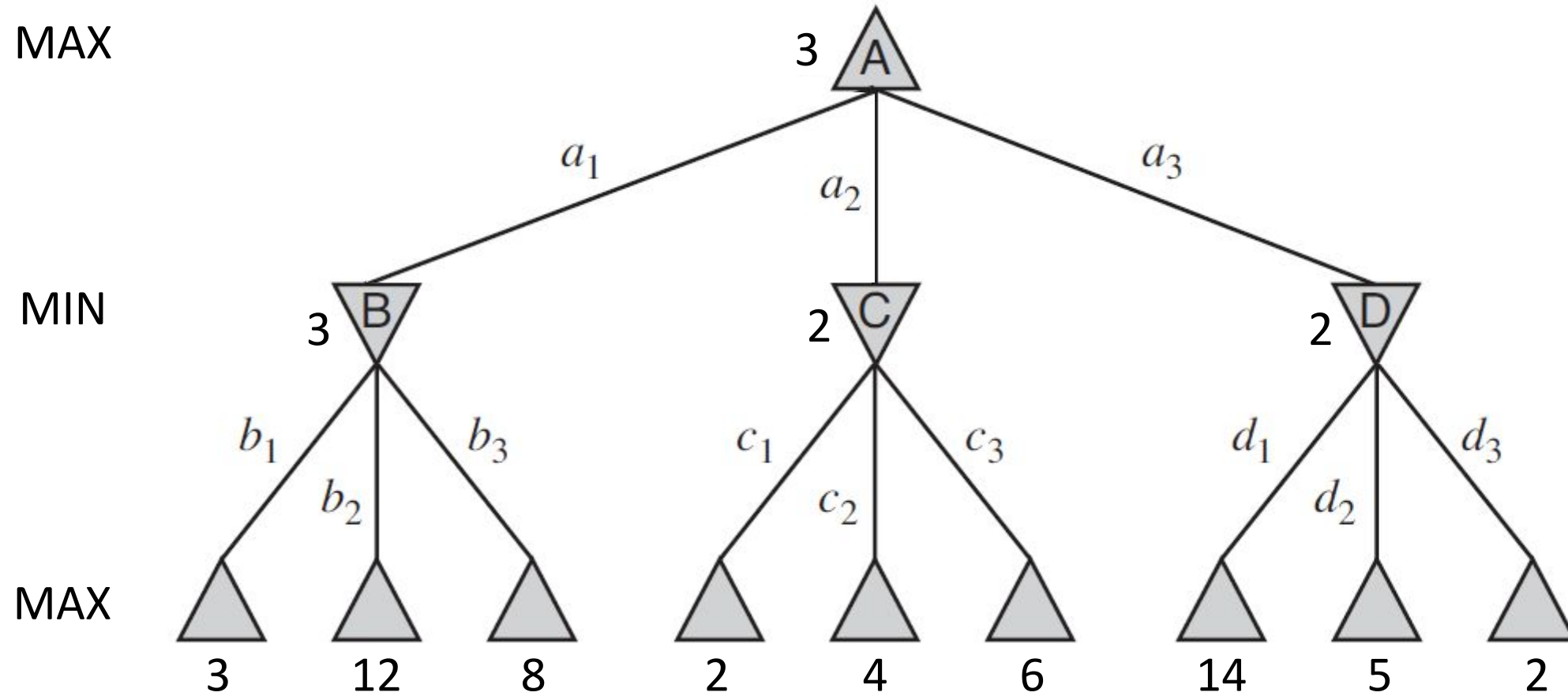
$v2, a2 \leftarrow \text{MAX-VALUE}(\text{game, game.RESULT}(\text{state}, a))$

if $v2 < v$ **then**

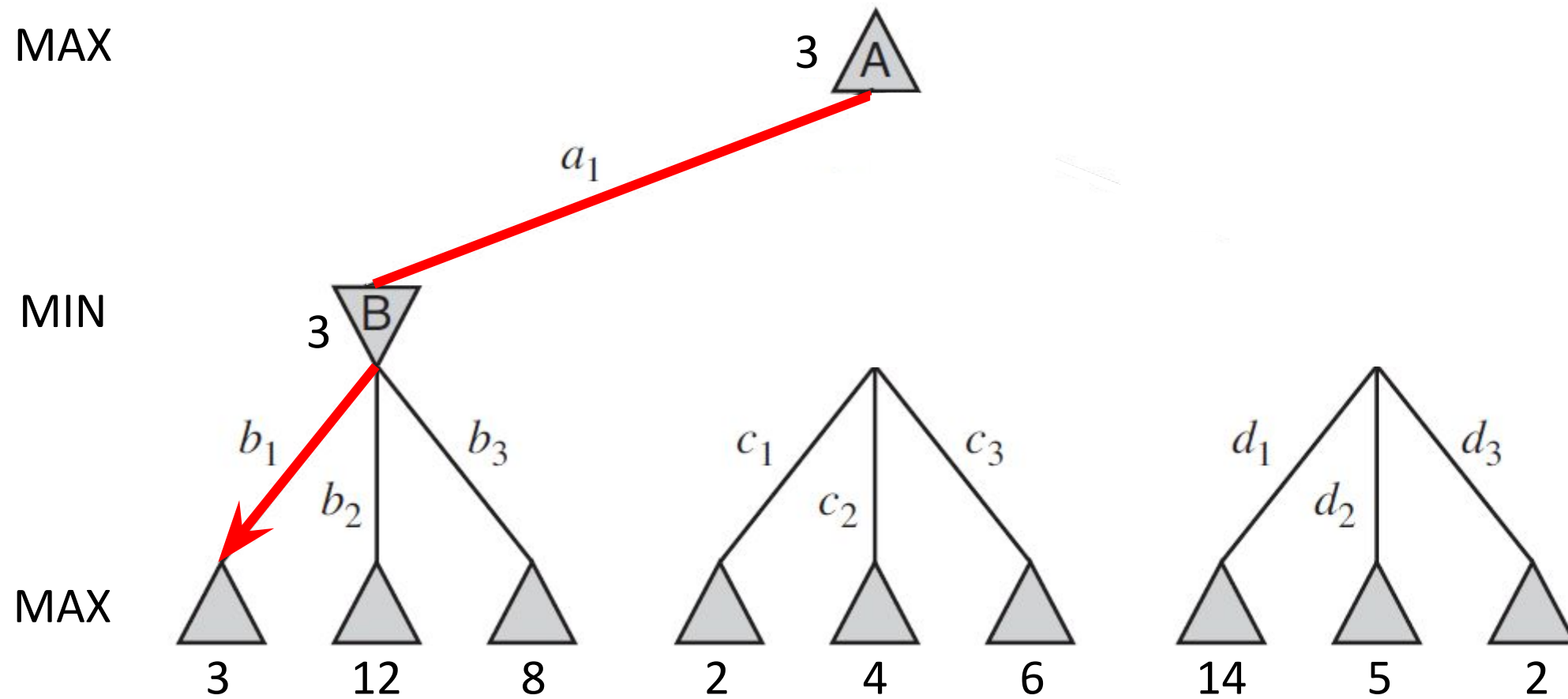
$v, \text{move} \leftarrow v2, a$

return v, move

The minimax Search



The minimax Search

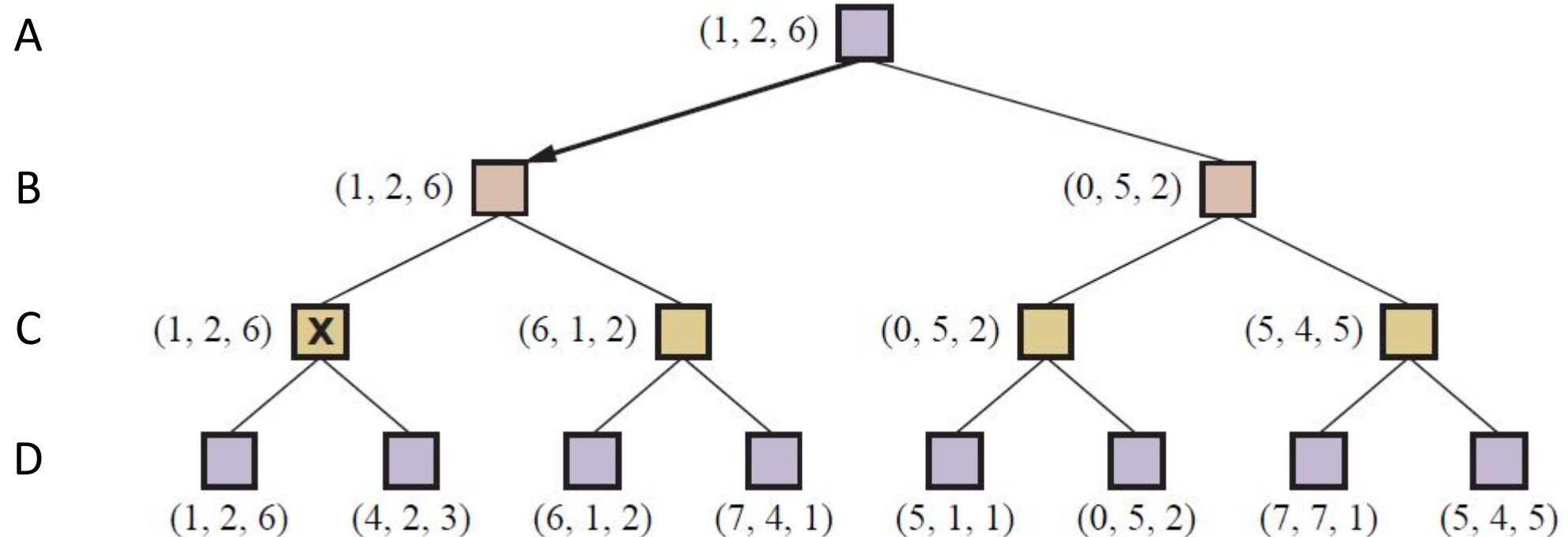


The minimax Search: Analysis

- Completeness:
 - A complete depth-first search exploration.
- Space complexity:
 - $O(bm)$ for an algorithm that generates all actions at once, and
 - $O(m)$ for an algorithm that generates actions one at a time where m is the maximum depth of the tree and b is legal moves at each point.
- Time complexity:
 - $O(b^m)$, the exponential complexity makes it impractical for complex games.
 - Example: Average chess game
 - It has to search for $35^{80} \approx 10^{123}$ states, where $b = 35$ and $m = 80$.

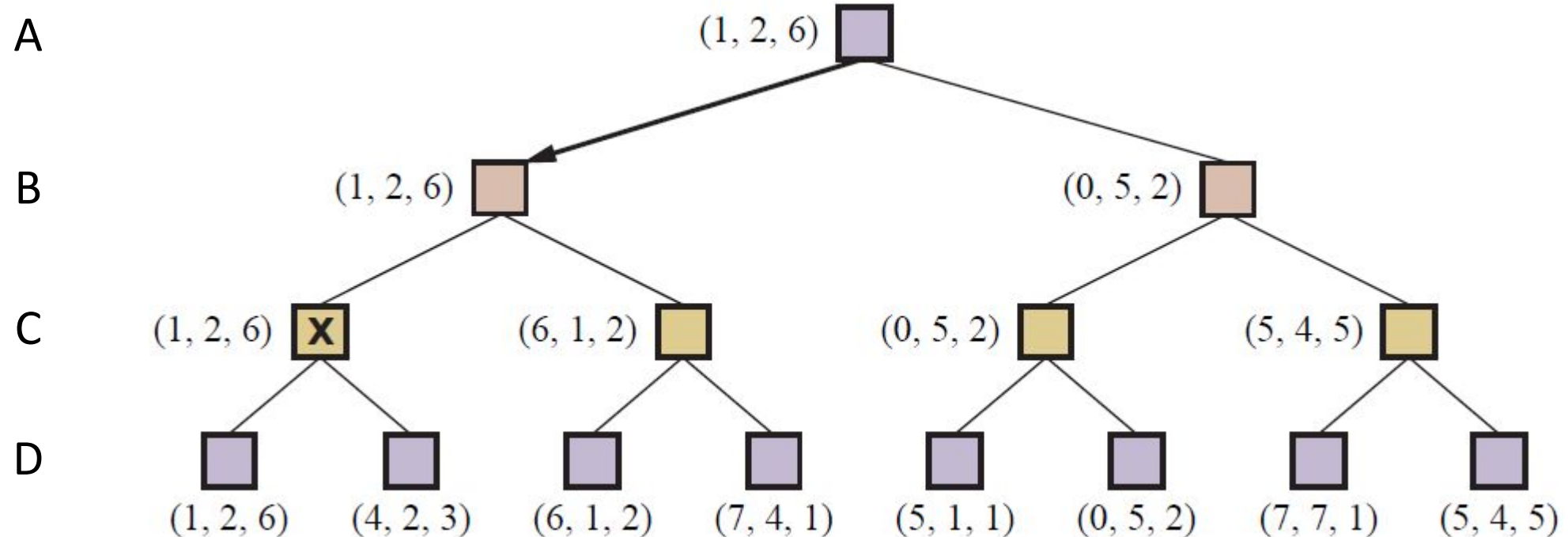
Optimal Decisions in Multiplayer Games

- Many popular games allow more than two players.
- The single value for each node has to be replaced with a **vector of values**.



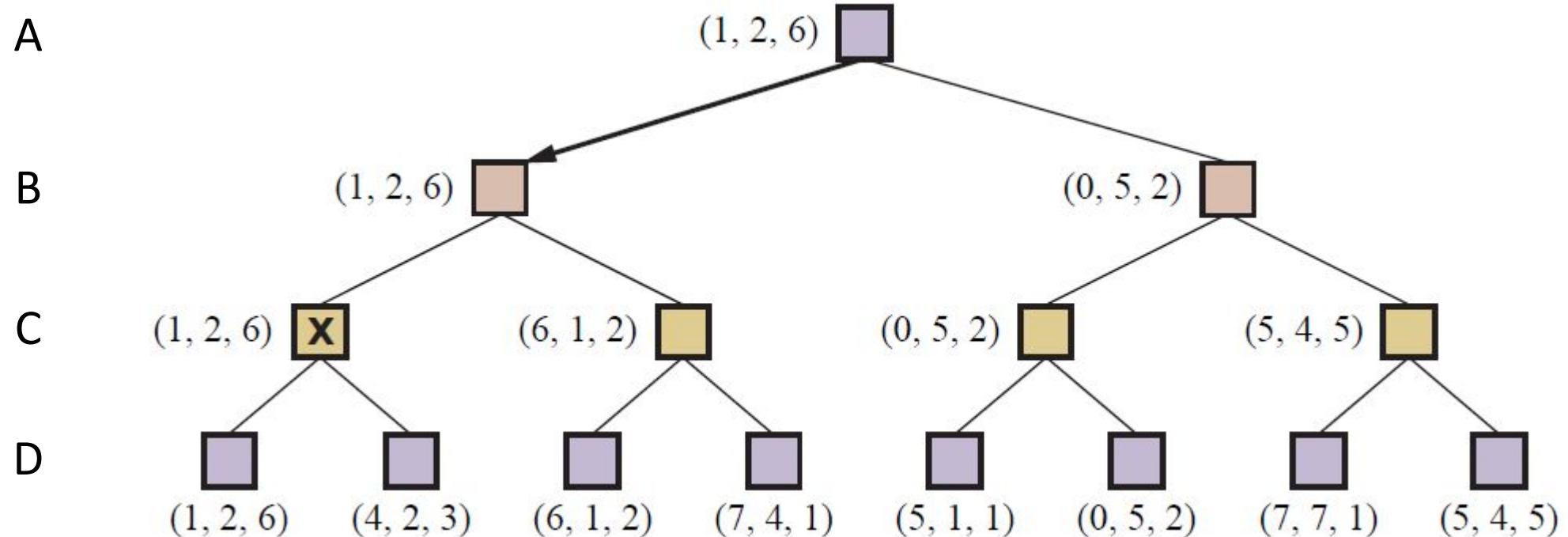
Optimal Decisions in Multiplayer Games

- For a three-player game with players A, B, and C, a vector $\langle v_A, v_B, v_C \rangle$ is associated with each node.



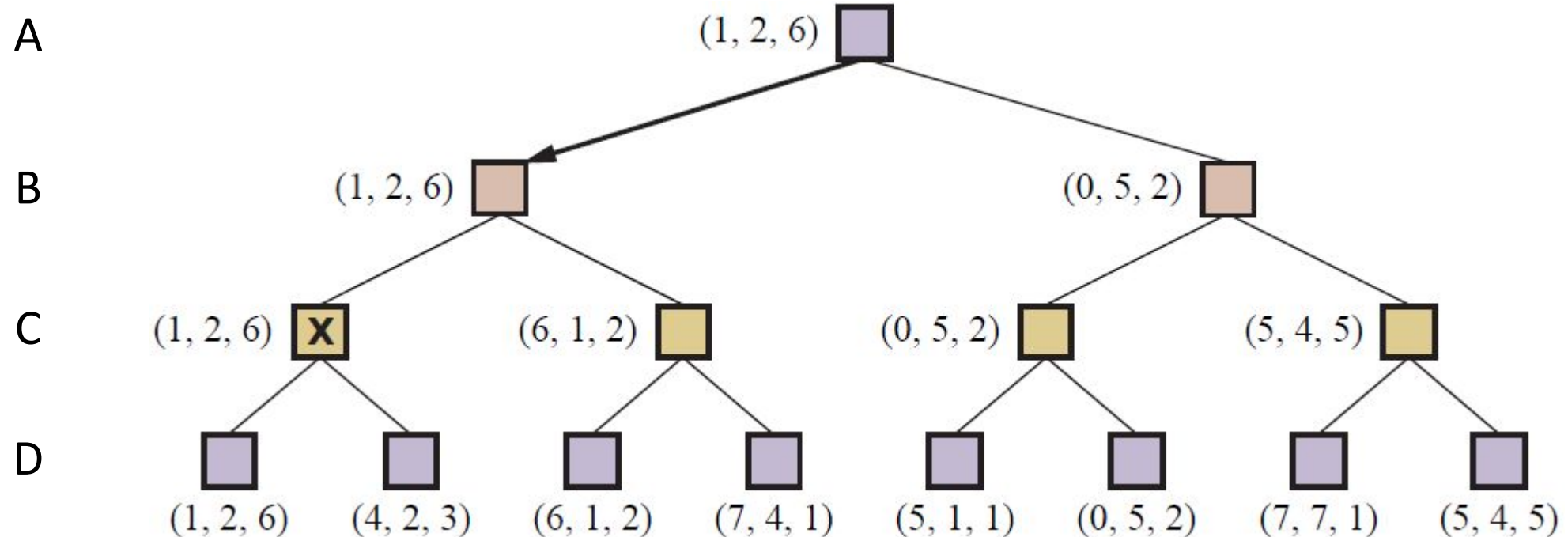
Optimal Decisions in Multiplayer Games

- For terminal states, the vector gives the utility of the state from each player's viewpoint.



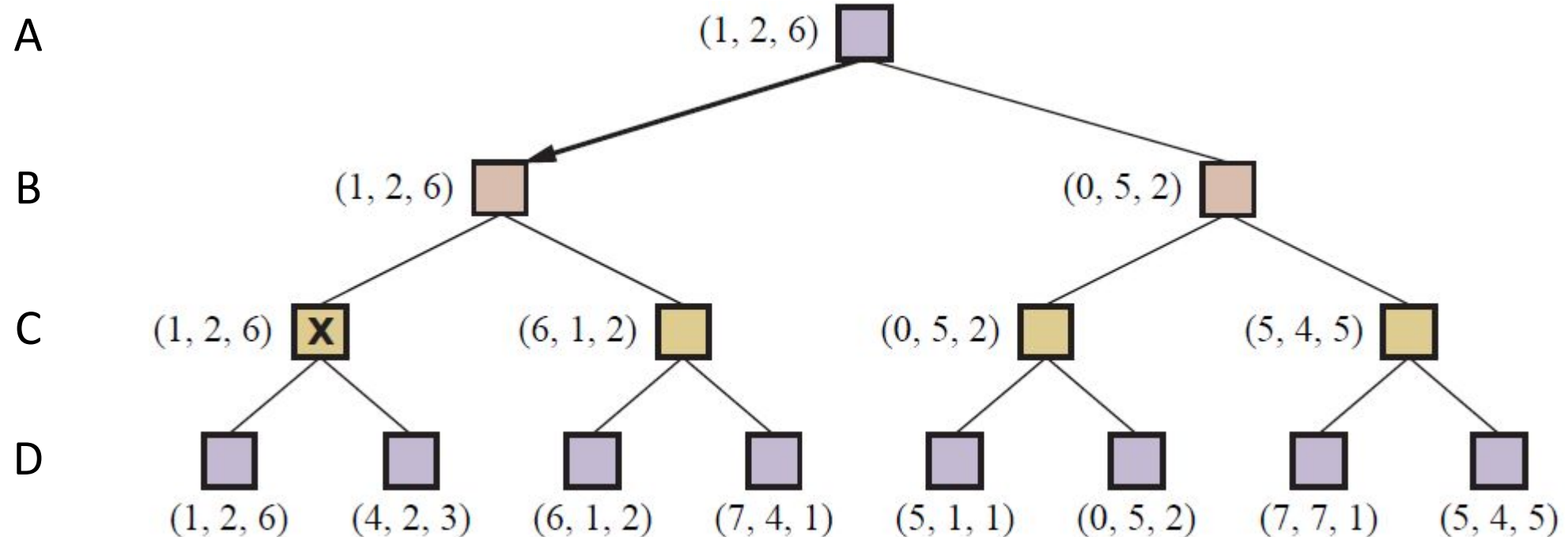
Optimal Decisions in Multiplayer Games

- For non-terminal states, the **backed-up value** of a node n is always the utility vector of the successor state with the highest value for the player choosing at n .



Optimal Decisions in Multiplayer Games

- Consider node X, there are two choices lead to terminal states with utility vectors $\langle v_A = 1, v_B = 2, v_C = 6 \rangle$ and $\langle v_A = 4, v_B = 2, v_C = 3 \rangle$, which one should C choose?



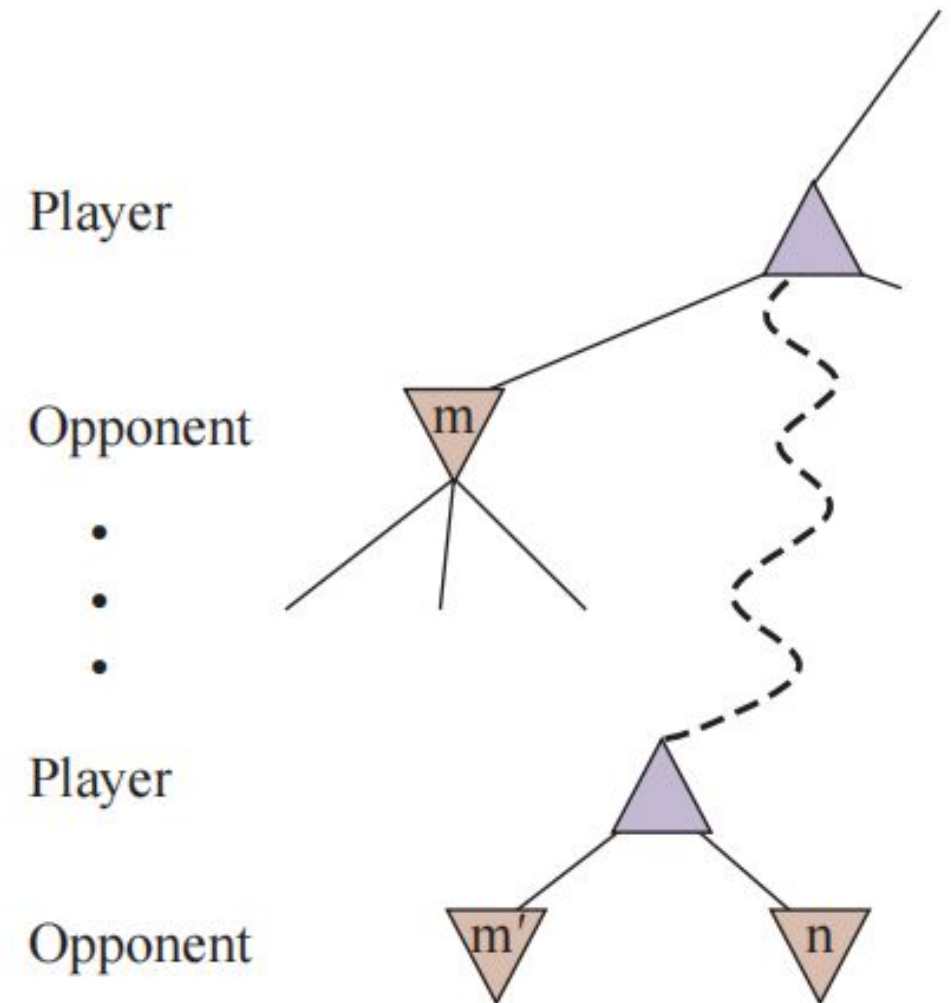
Alpha-Beta Pruning

- The number of game states minimax search has to examine is exponential in the depth of the tree.
- With **alpha-beta pruning**, the gigantic number can be reduced by almost half.
- When it is applied to a standard minimax tree, it
 - Returns the same move as minimax would.
 - But prunes away branches that **cannot possibly influence the minimax decision**.
- Alpha-beta pruning can be applied to trees of any depth, and it is often possible to **prune entire subtrees** rather than just leaves.

Alpha-Beta Pruning

□ General principle

- Consider a node n somewhere in the tree such that Player has a choice of moving to n .
- If the player has a better choice at the same level (e.g., m) or any point higher up in the tree (e.g., m'), then Player will never move to n .
- When this happens, n can be pruned.



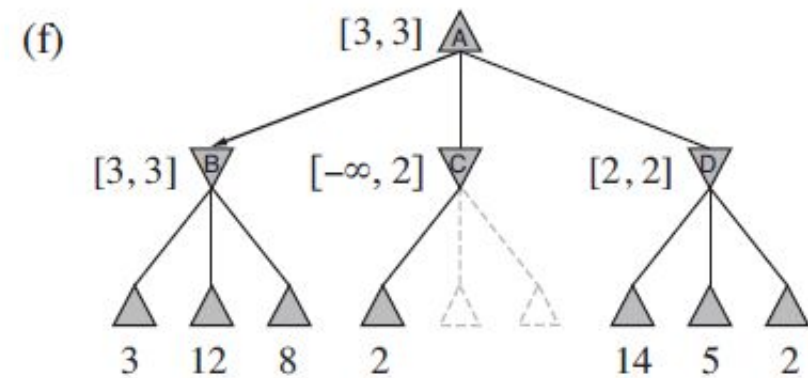
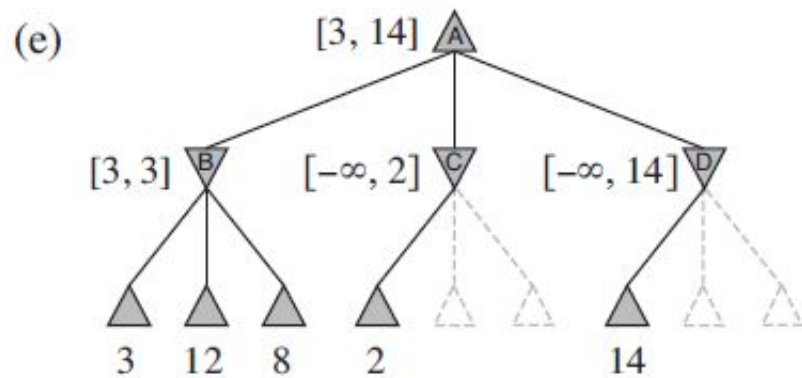
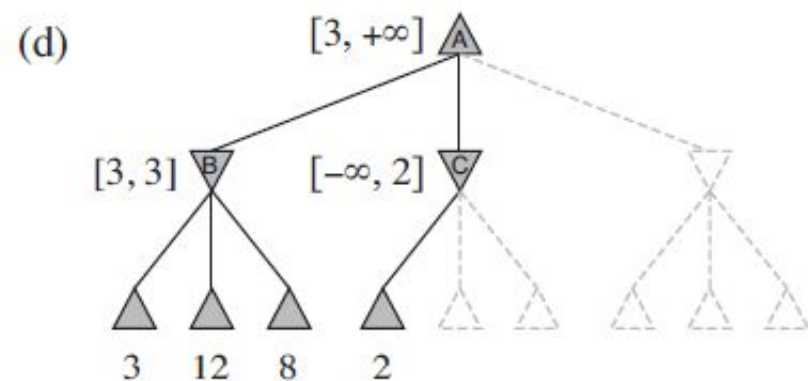
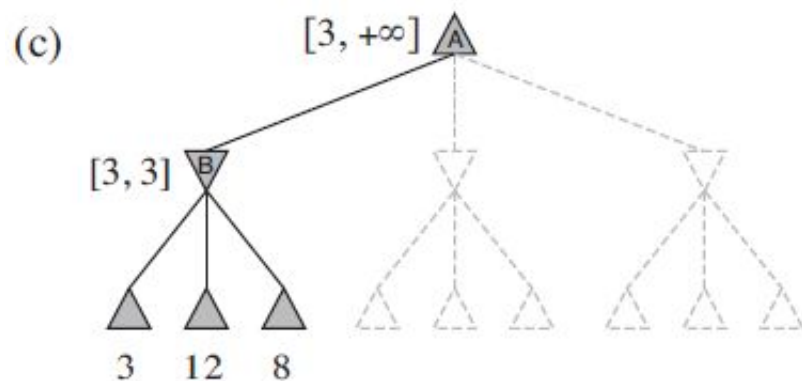
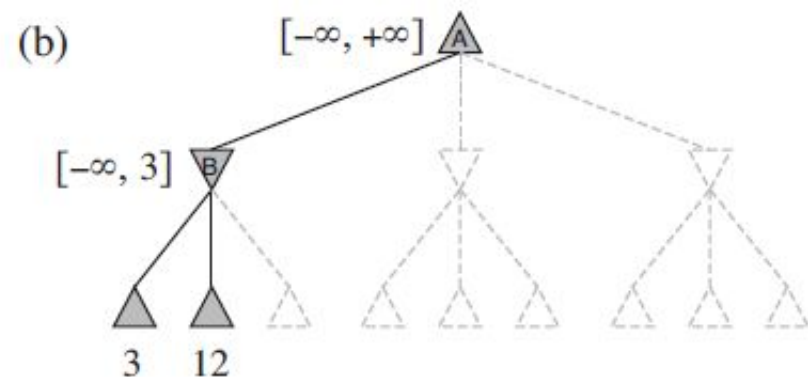
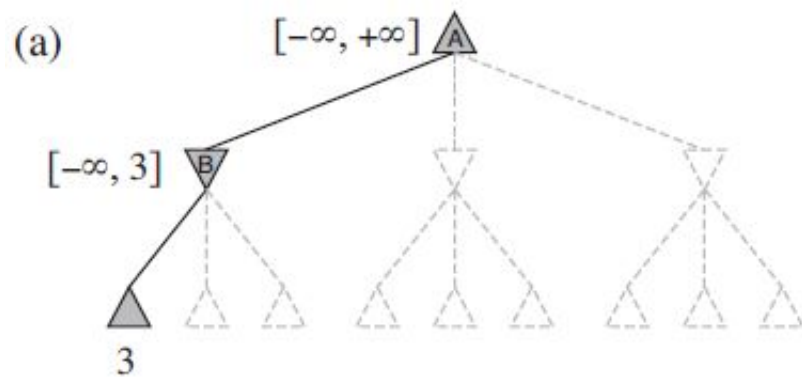
Alpha-Beta Pruning

- Alpha-beta pruning gets its name from two bounds that are passed along during the calculation:
 - α = the best highest-value choice found so far along the path for MAX.
 - β = the best lowest-value choice found so far along the path for MIN.
- Given N as the current estimate of the value of any node, it is only considered as a possible path to the solution if:

$$\alpha \leq N \leq \beta$$

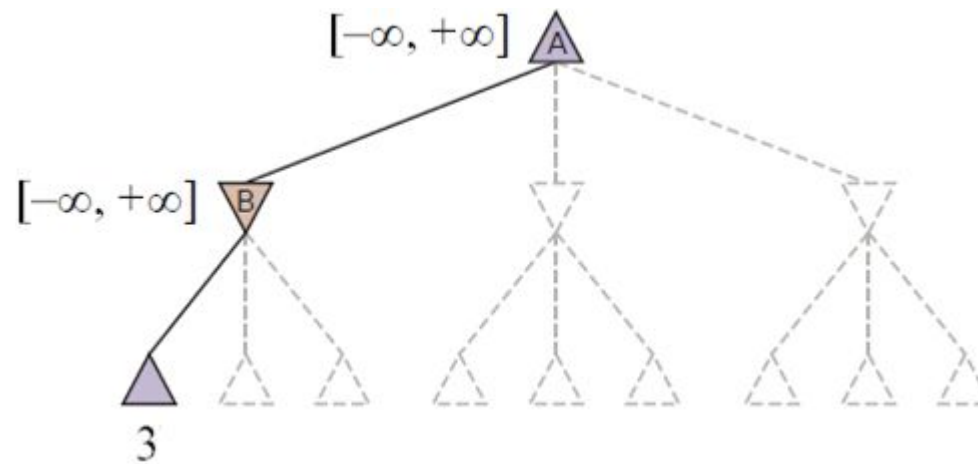
- At any point of node, when $\alpha > \beta$, it means that **this node could never result in a solution path**, and hence, **its remaining successors will be pruned**.

Alpha-Beta Pruning



Alpha-Beta Pruning

- At the start of the problem, the value of α and β for the current state is a number greater than negative infinity and less than infinity respectively.

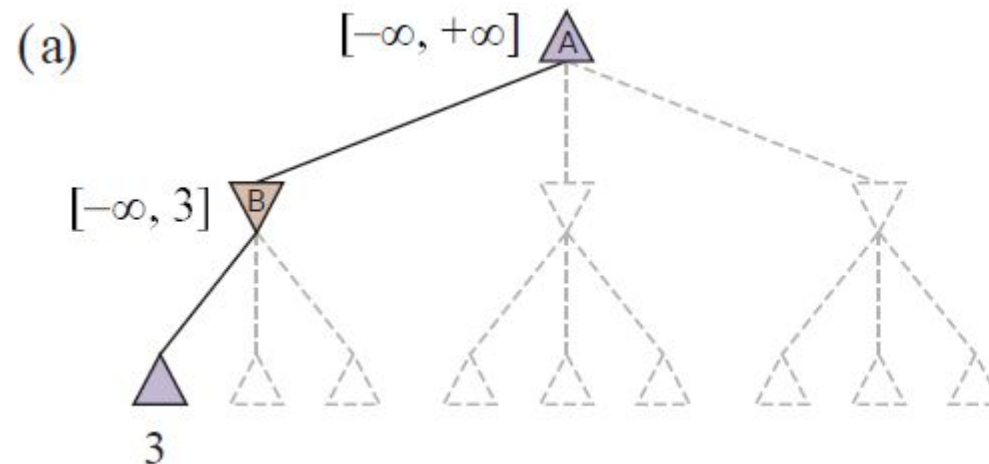


- The initial bounds will be passed down to its child node until it reaches a terminal node where the calculation of the optimal decision begins.

Alpha-Beta Pruning

□ Step (a)

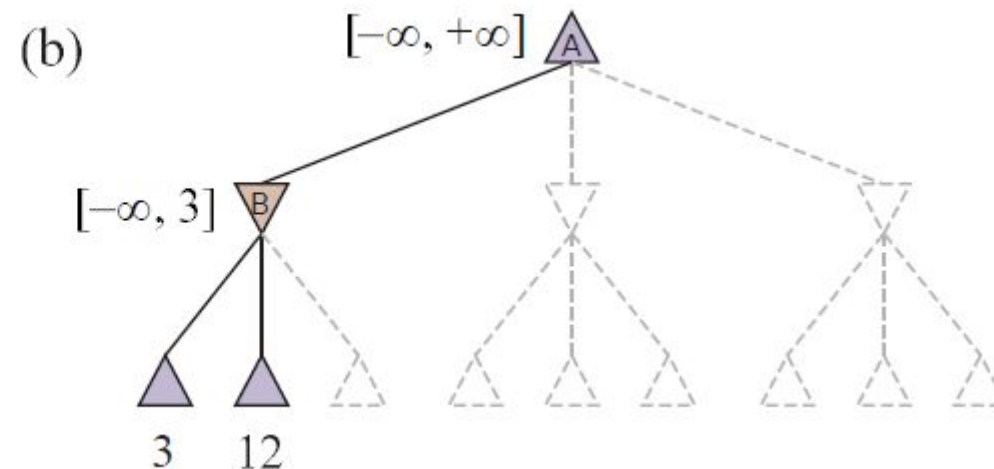
- The first leaf below B has the value 3.
- B, as a MIN node, has a value of **at most 3**.
- The value of 3 is backed-up at B, replacing $+\infty$.



Alpha-Beta Pruning

□ Step (b)

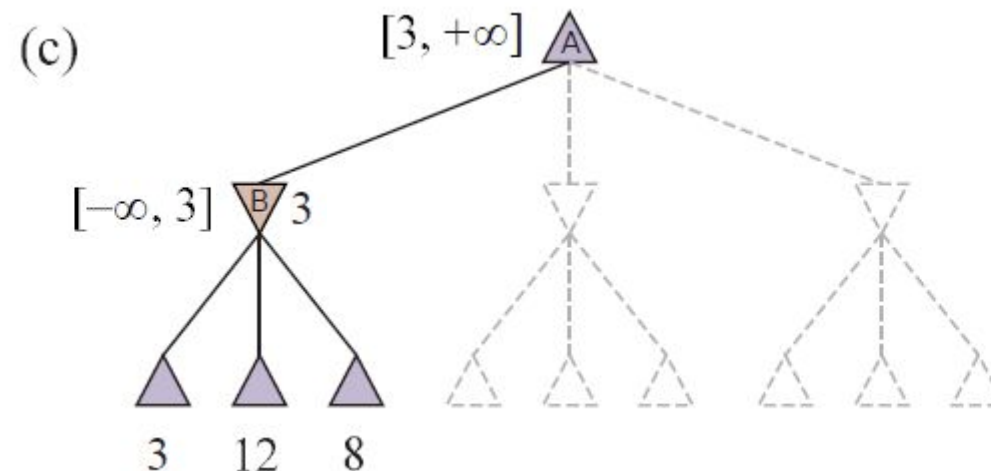
- The second leaf below B has a value of 12.
- MIN would avoid this move, so, the value of B is still at most 3.



Alpha-Beta Pruning

□ Step (c)

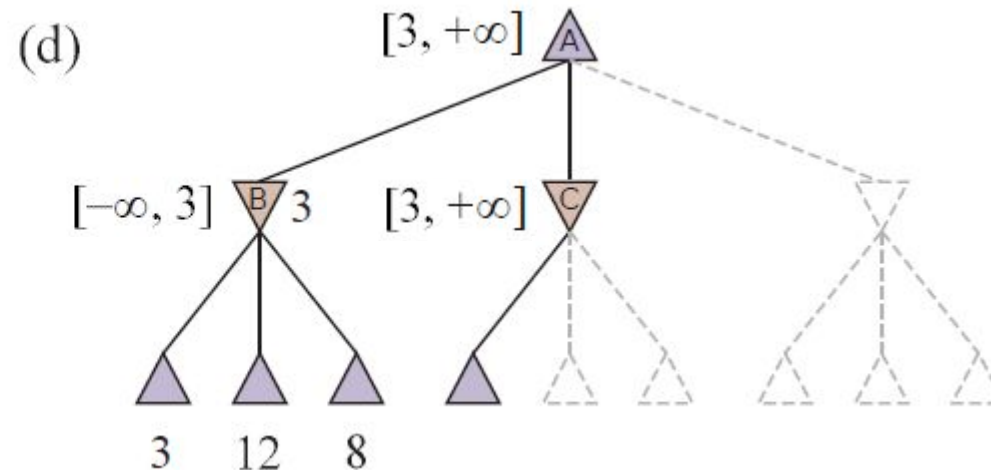
- The third leaf below B has a value of 8 (all B's successor states are known).
- For the same reason, the value of B is still 3.
- Since now MAX has a choice worth 3, the value of the root is **at least 3**.



Alpha-Beta Pruning

□ Step (d1):

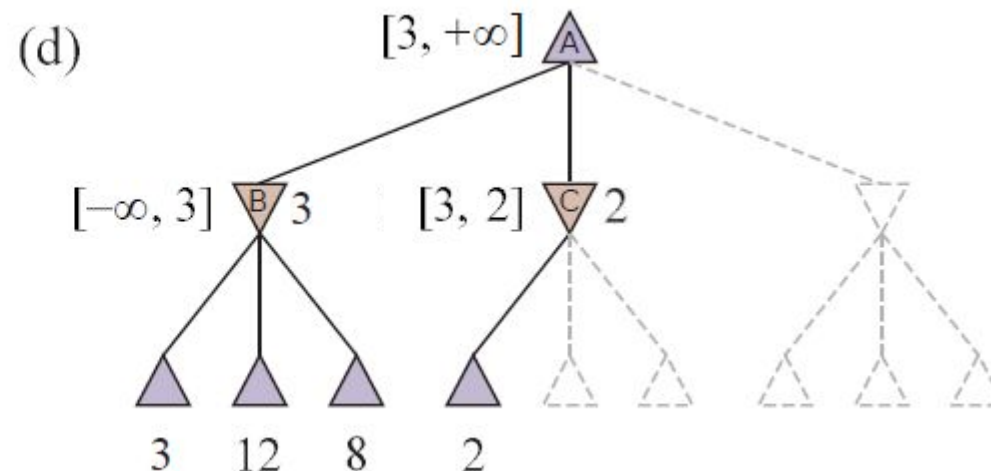
□ Now, MIN will explore node C's successors.



Alpha-Beta Pruning

□ Step (d2):

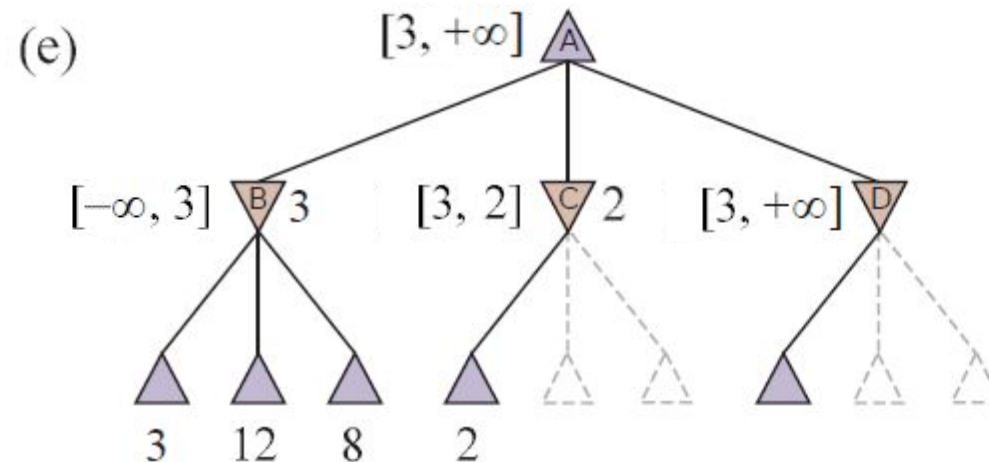
- The first leaf below C has the value 2, and hence B has a value of **at most 2**.
- But B is worth 3 (now at node C, $\alpha > \beta$), so MAX would never choose C.
- Other successor states of C will not be explored and pruned.



Alpha-Beta Pruning

□ Step (e1):

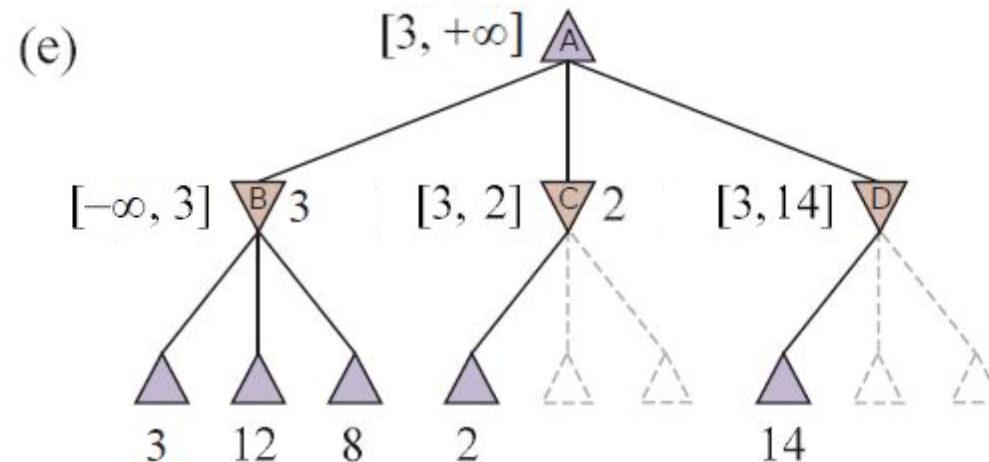
□ Now, MIN will explore node D's successors.



Alpha-Beta Pruning

□ Step (e2):

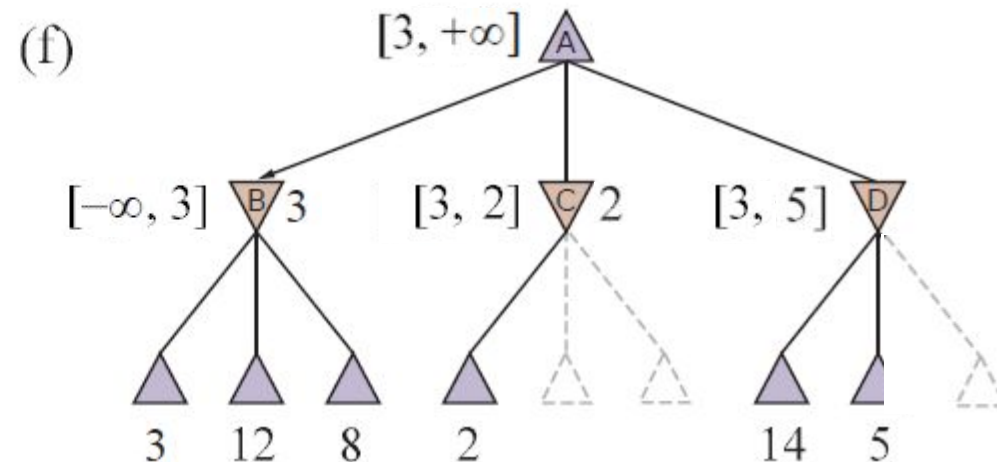
- The first leaf below D has the value 14, so D is worth at most 14.
- This is higher than MAX's best alternative (i.e., 3), so exploration continues.
- All of the root's successors have bounds now, so the root's value is at most 14.



Alpha-Beta Pruning

□ Step (f):

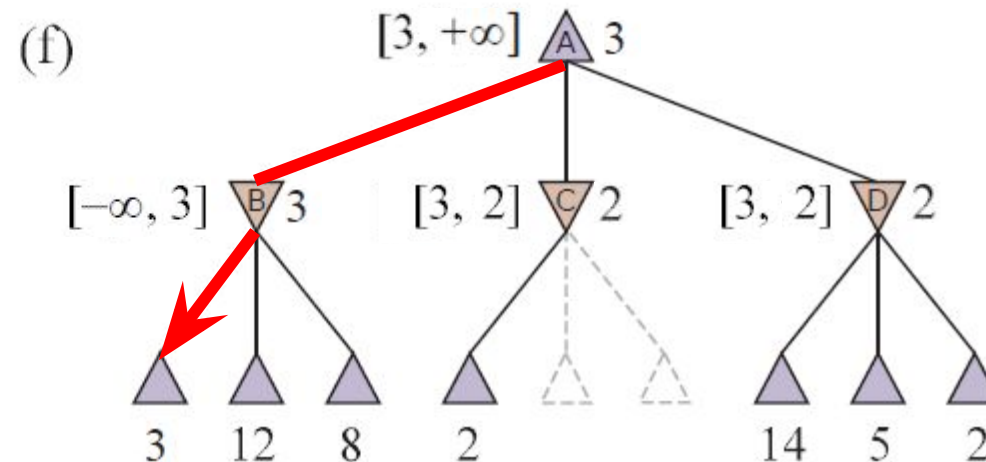
- The second successor of D is worth 5.
- Since 5 is a better option for MIN, the backed-up value of 14 is replaced with 5.



Alpha-Beta Pruning

□ Step (g):

- The third successor is worth 2, so now D is worth exactly 2.
- MAX's decision at the root is to move to B, giving a value of 3.



Alpha-Beta Pruning

- Another way to look at alpha-beta pruning: a simplification of the formula for MINIMAX.
- Let the two unevaluated successors of node C have values **x** and **y**.
- Then the value of the root node is given by

$$\begin{aligned}\text{MINIMAX}(\text{root}) &= \max(\min(3, 12, 8), \min(2, x, y), \min(14, 5, 2)) \\ &= \max(3, \min(2, x, y), 2) \\ &= \max(3, z, 2) \quad \text{where } z = \min(2, x, y) \leq 2 \\ &= 3\end{aligned}$$

- Thus, the minimax decision is independent of the pruned leaves **x** and **y**.

function ALPHA-BETA-SEARCH(*game*, *state*) **returns** an action

$\text{player} \leftarrow \text{game.TO-MOVE}(\text{state})$

$\text{value}, \text{move} \leftarrow \text{MAX-VALUE}(\text{game}, \text{state}, -\infty, +\infty)$

return *move*

function MAX-VALUE(*game*, *state*, α , β) **returns** a (*utility*, *move*) pair

if *game.IS-TERMINAL*(*state*) **then return** *game.UTILITY*(*state*, *player*), *null*

$v \leftarrow -\infty$

for each *a* **in** *game.ACTIONS*(*state*) **do**

$v_2, a_2 \leftarrow \text{MIN-VALUE}(\text{game}, \text{game.RESULT}(\text{state}, a), \alpha, \beta)$

if $v_2 > v$ **then**

$v, \text{move} \leftarrow v_2, a$

$\alpha \leftarrow \text{MAX}(\alpha, v)$

if $v \geq \beta$ **then return** *v*, *move*

return *v*, *move*

function MIN-VALUE(*game*, *state*, α , β) **returns** a (*utility*, *move*) pair

if *game.IS-TERMINAL*(*state*) **then return** *game.UTILITY*(*state*, *player*), *null*

$v \leftarrow +\infty$

for each *a* **in** *game.ACTIONS*(*state*) **do**

$v_2, a_2 \leftarrow \text{MAX-VALUE}(\text{game}, \text{game.RESULT}(\text{state}, a), \alpha, \beta)$

if $v_2 < v$ **then**

$v, \text{move} \leftarrow v_2, a$

$\beta \leftarrow \text{MIN}(\beta, v)$

if $v \leq \alpha$ **then return** *v*, *move*

return *v*, *move*

Reading Homework

- **Artificial Intelligence: A Modern Approach**, 4th Edition, by Stuart Russell and Peter Norvig
 - Chapter 6: Adversarial Search (page 192 to page 199)

The End